Практическая работа №1

Пользовательские запросы в веб-приложении (DevTools, HTTP, REST, JSON, Postman)

1.Теория:

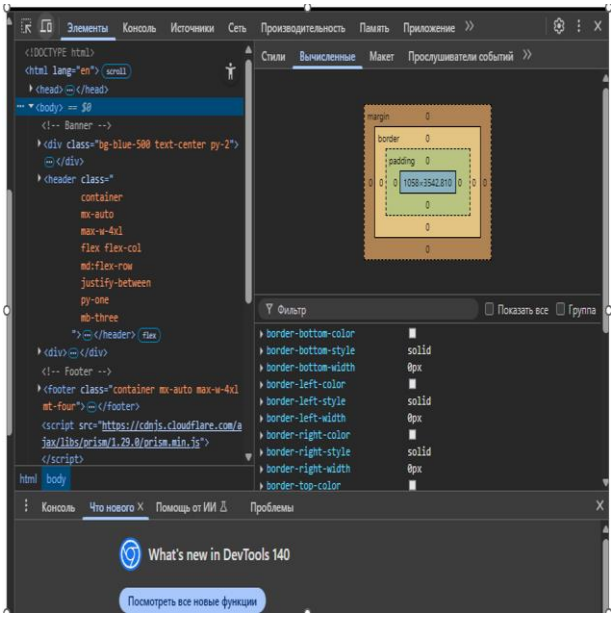
1.1. Что такое DevTools

DevTools (Developer Tools) – встроенные инструменты разработчика браузера (Chrome, Firefox, Edge, Safari).
Открытие: **F12**.

Основные панели:

- **Elements –** DOM и визуальная структура страницы.
- **Console –** ошибки, логи, выполнение JS.
- **Network –** анализ запросов, коды, заголовки, время.
- **Performance –** измерение скорости работы страницы.
- **Device mode –** проверка адаптивности интерфейса.
- **Security –** HTTPS, сертификаты.
- **Memory –** утечки памяти и использование ресурсов.

Пример интерфейса DevTools:



1.2. Пользовательские запросы

Пользовательские запросы – это **HTTP-запросы** (GET, POST и др.), которые браузер отправляет при работе с веб-приложением.

Основные характеристики запроса:

- метод (GET/POST/PUT/DELETE/…)
- URL и параметры
- заголовки (headers)
- тело запроса / ответа
- время выполнения
- код ответа

Оптимизация запросов:

- уменьшение количества обращений;
- сокращение объёма передаваемых данных;
- кэширование;
- пагинация и фильтрация;
- использование правильных HTTP-методов и заголовков.

Основные HTTP-методы

Метод	CRUD	Назначение	Особенности	Пример
GET	Read	Получение данных	Параметры в URL, кэшируемый, не меняет данные	`/users?id=5`
POST	Create	Создание ресурса	Данные в теле, изменяет состояние	Регистрация, заказ
PUT	Update	Полное обновление	Заменяет весь объект	Обновить профиль
PATCH	Update	Частичное обновление	Меняет только указанные поля	Сменить только email
DELETE	Delete	Удаление ресурса	Обычно `200 OK` или `204 No Content`	Удалить товар

HEAD	–	Только заголовки	Проверка наличия / размера / даты	Проверка `Last-Modified`

1.3. Коды ответов

- **1xx** – информационные (например, `100 Continue`).
- **2xx** – успешные (`200 OK`, `201 Created`, `204 No Content`).
- **3xx** – перенаправления (`301`, `302`, `304 Not Modified`).
- **4xx** – ошибки клиента (`400`, `401`, `403`, `404`).
- **5xx** – ошибки сервера (`500`, `502`, `503`, `504`).

1.4. REST API

REST – архитектурный стиль взаимодействия с ресурсами по HTTP.

Принципы:

- клиент–сервер;
- отсутствие состояния (stateless);
- единообразный интерфейс;
- работа с ресурсами (уникальные URL);
- часто используется формат **JSON**.

1.5. JSON

JSON – лёгкий текстовый формат обмена данными (объекты, массивы, строки, числа, boolean, `null`).

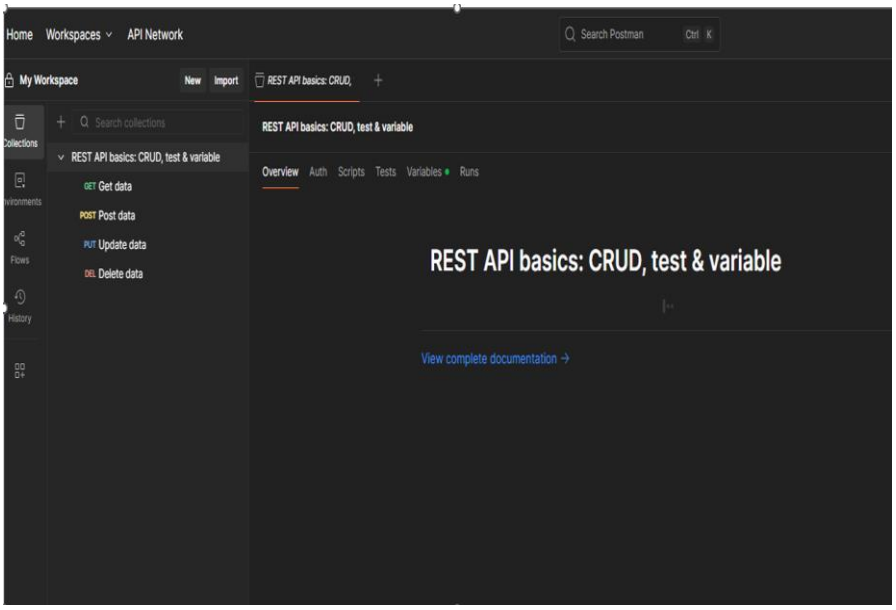
Пример ответа:

```
{
  "userId": 1,
  "id": 1,
  "title": "sunt aut facere...",
  "body": "quia et suscipit..."
}
```

1.6. Postman

Postman – инструмент для работы с API:

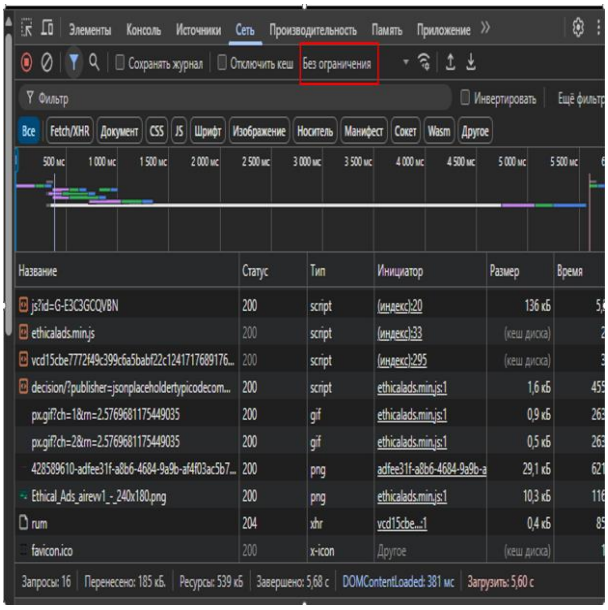
- создание и отправка запросов;
- анализ ответов;
- сохранение коллекций;
- автотесты;
- совместная работа.



2. Задание

2.1. Требуемые инструменты

- браузер (Chrome / Firefox / Edge / Яндекс.Браузер);
- DevTools → вкладка **Network** (режим Online, без ограничения скорости или с 4G-профилем).



2.2. Первичный анализ запросов

1. Откройте DevTools (**F12**) → вкладка **Network**.
2. Загрузите страницу:

```
https://jsonplaceholder.typicode.com/posts
```

1. Зафиксируйте в таблице: метод, URL, код ответа, размер данных, время загрузки.



2. Сделайте скриншот списка запросов.

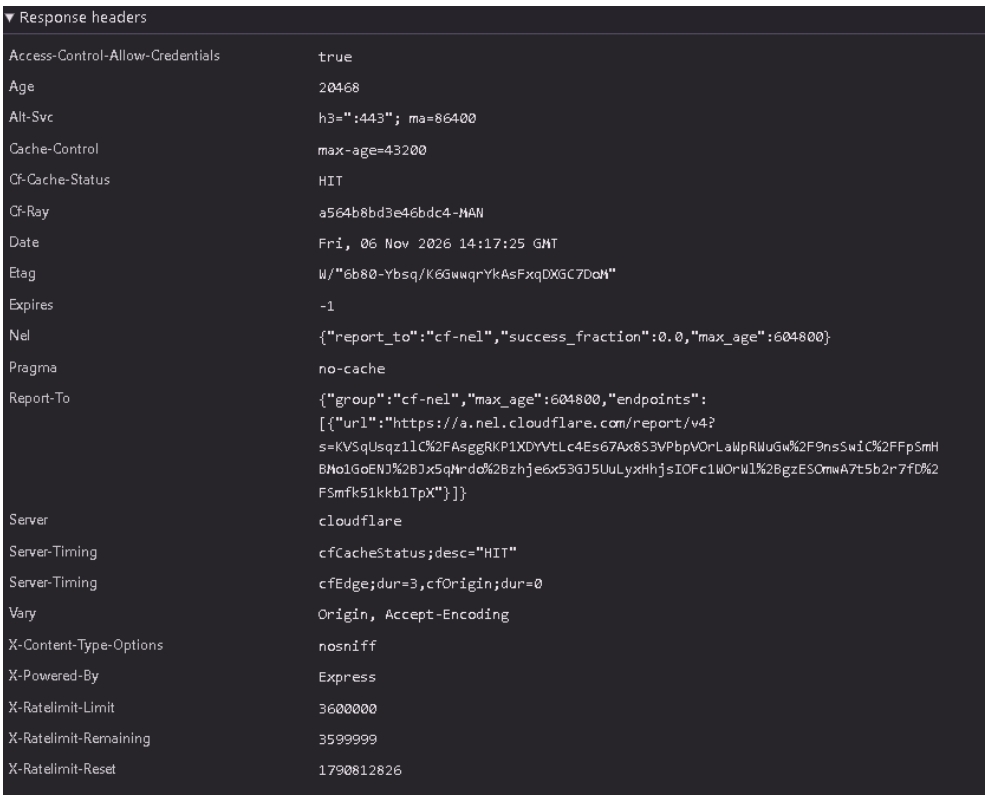


2.3. Детальный разбор одного запроса

Выберите любой запрос и опишите:

- Метод – Get
- URL и параметры – https://jsonplaceholder.typicode.com/posts, параметров нет;
- заголовки (Request Headers / Response Headers);

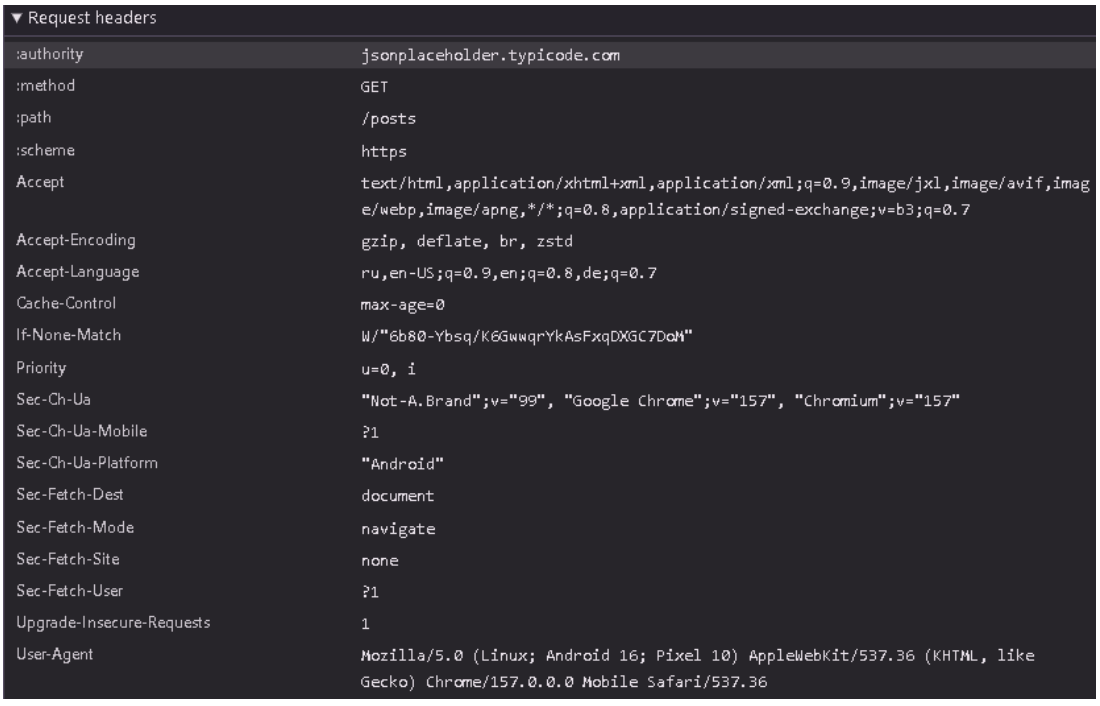
Response Headers:



▼ Response headers

Access-Control-Allow-Credentials	true
Age	20468
Alt-Svc	h3=":443"; ma=86400
Cache-Control	max-age=43200
Cf-Cache-Status	HIT
Cf-Ray	a564b8bd3e46bdc4-MAN
Date	Fri, 06 Nov 2026 14:17:25 GMT
Etag	W/"6b80-Ybsq/K6GwwqrYkAsFxqDXGC7DoM"
Expires	-1
Nel	{"report_to":"cf-nel","success_fraction":0.0,"max_age":604800}
Pragma	no-cache
Report-To	{"group":"cf-nel","max_age":604800,"endpoints":[{"url":"https://a.nel.cloudflare.com/report/v4?s=KVSqUsqz1lC%2FAsggRKP1XDYVtLc4Es67Ax8S3VPbpVOrLaWpRWuGw%2F9nsSwiC%2FFpSmHBMo1GoENJ%2BJx5qMrdo%2Bzhje6x53GJ5UuLyxHhjsIOFc1WOrWl%2BgzESOmwA7t5b2r7fD%2FSmfk51kkb1TpX"}]}
Server	cloudflare
Server-Timing	cfCacheStatus;desc="HIT"
Server-Timing	cfEdge;dur=3,cfOrigin;dur=0
Vary	Origin, Accept-Encoding
X-Content-Type-Options	nosniff
X-Powered-By	Express
X-Ratelimit-Limit	3600000
X-Ratelimit-Remaining	3599999
X-Ratelimit-Reset	1790812826

Request Headers:



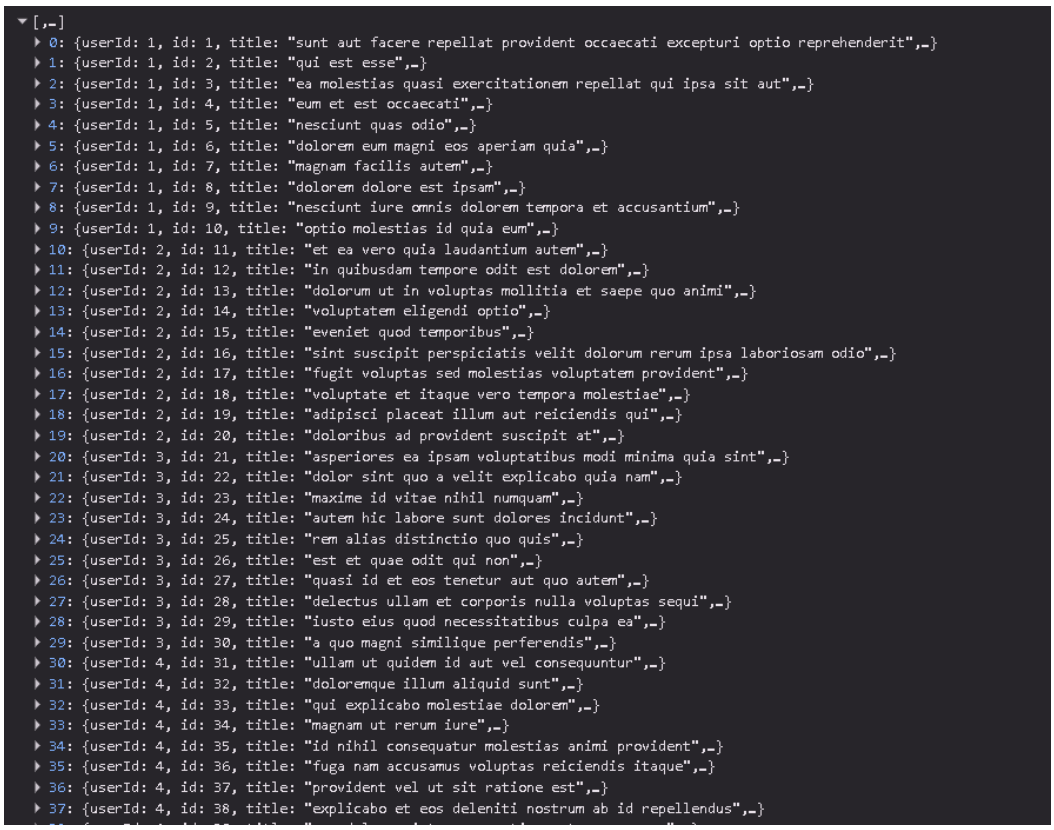
▼ Request headers

:authority	jsonplaceholder.typicode.com
:method	GET
:path	/posts
:scheme	https
Accept	text/html,application/xhtml+xml,application/xml;q=0.9,image/jxl,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Accept-Encoding	gzip, deflate, br, zstd
Accept-Language	ru,en-US;q=0.9,en;q=0.8,de;q=0.7
Cache-Control	max-age=0
If-None-Match	W/"6b80-Ybsq/K6GwwqrYkAsFxqDXGC7DoM"
Priority	u=0, i
Sec-Ch-Ua	"Not-A.Brand";v="99", "Google Chrome";v="157", "Chromium";v="157"
Sec-Ch-Ua-Mobile	?1
Sec-Ch-Ua-Platform	"Android"
Sec-Fetch-Dest	document
Sec-Fetch-Mode	navigate
Sec-Fetch-Site	none
Sec-Fetch-User	?1
Upgrade-Insecure-Requests	1
User-Agent	Mozilla/5.0 (Linux; Android 16; Pixel 10) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/157.0.0.0 Mobile Safari/537.36

- тело ответа (Response / Preview);

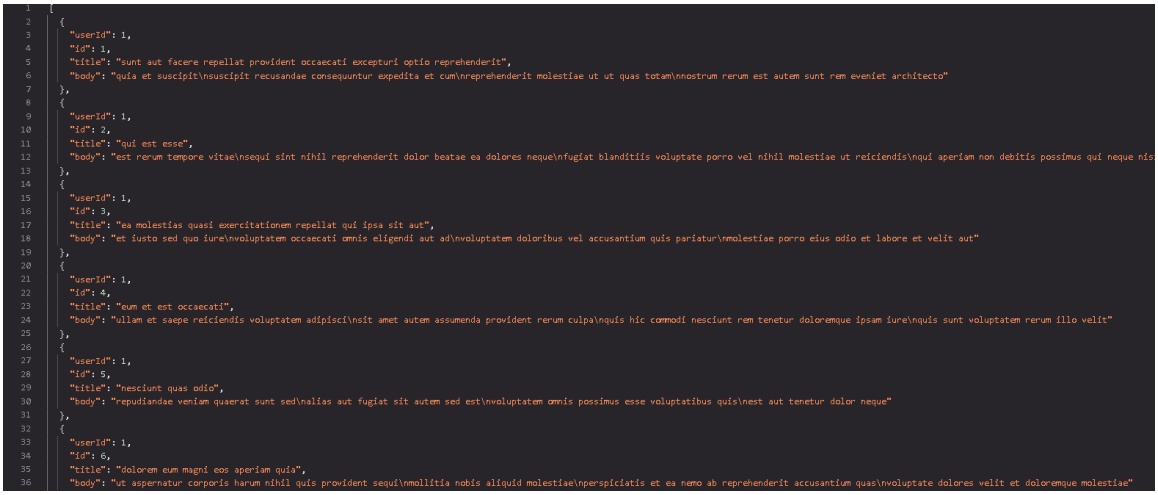
Тело ответа (Preview)

Массив из 100 объектов (id от 1 до 100):



Тело ответа (Response)

Один JSON-массив из 100 объектов:



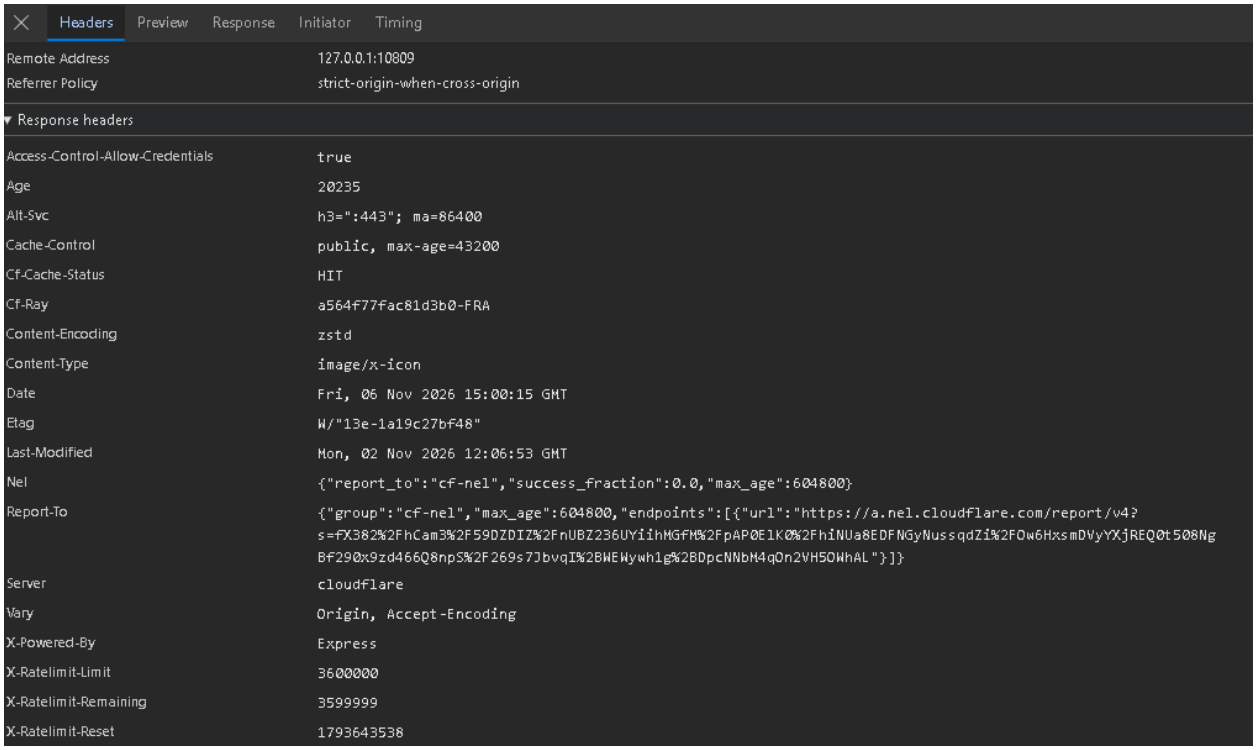
- код ответа – 304
- время выполнения – 350 ms

Сравните его с другим запросом и сделайте вывод.

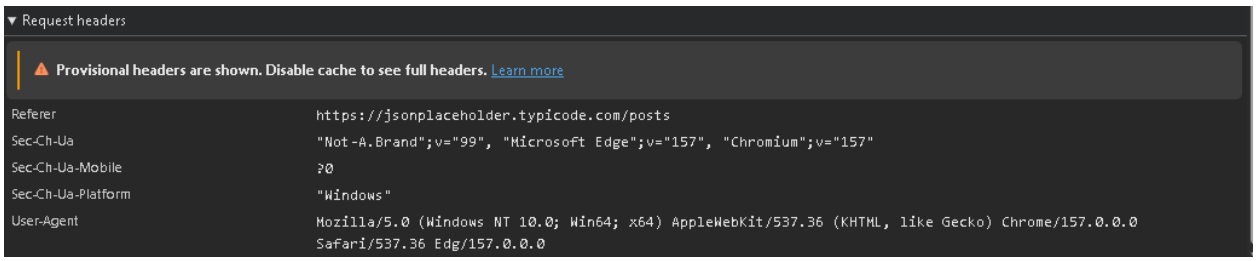
- Метод – Get
- URL и параметры - jsonplaceholder.typicode.com/posts, параметров нет;

- заголовки (Request Headers / Response Headers);

Response Headers:

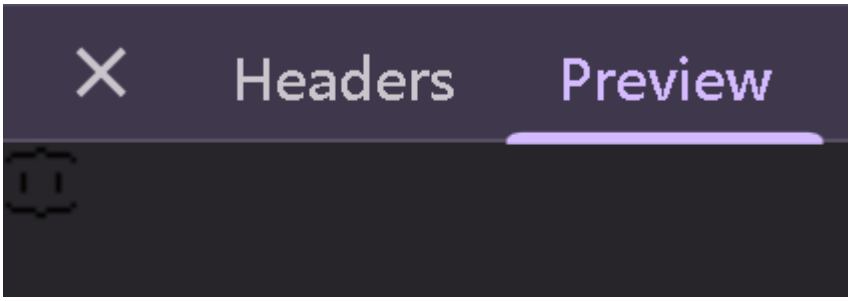


Request Headers:

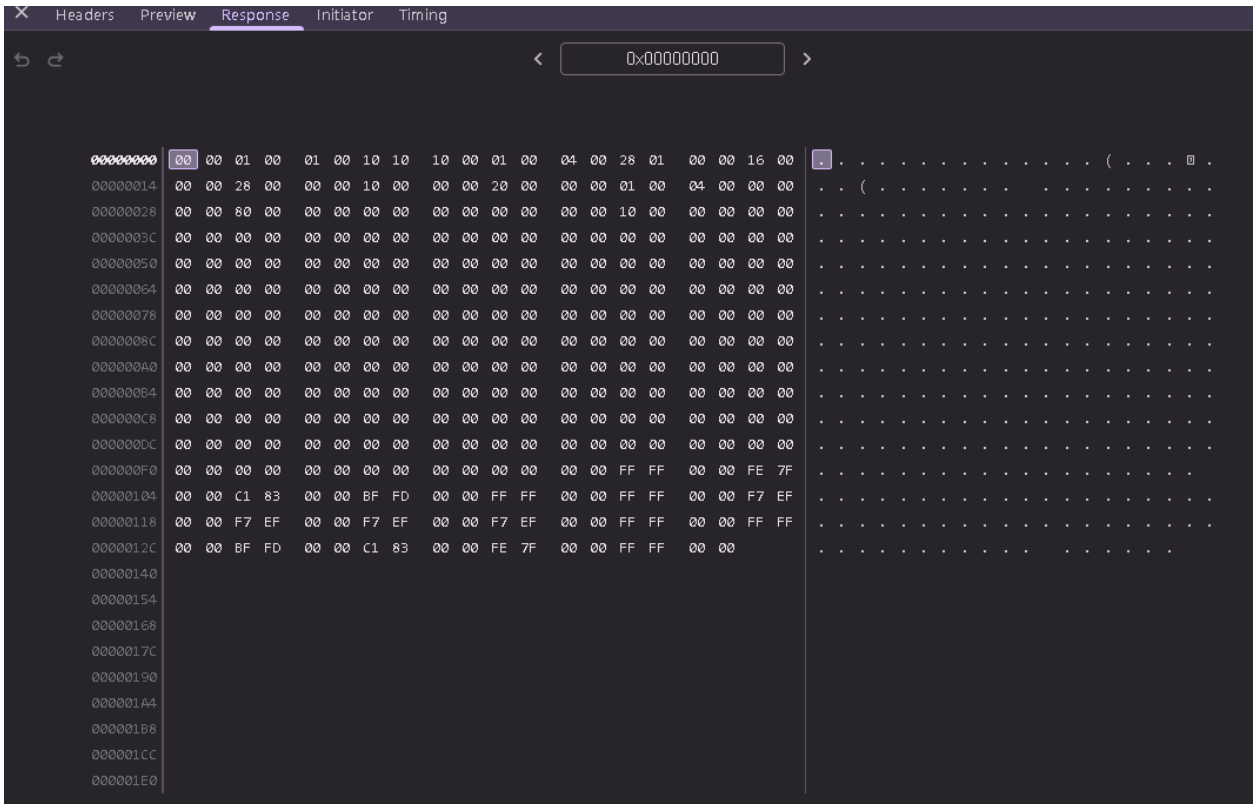


- тело ответа (Response / Preview);

Тело ответа (Preview)



Тело ответа (Response)



- код ответа – 200
- время выполнения – 2 ms

Вывод: одно картинка, другое массив JSON, с одинаковыми методами, разные коды ответа, иконку всегда заново загружает при обновлении, а JSON файл просто храниться в памяти и при обновлении просто возвращается.

2.4. Эксперименты с запросами

Выполните в консоли:

```
fetch("https://jsonplaceholder.typicode.com/posts/1")
  .then(res => res.json())
  .then(data => console.log(data));
```

Проанализируйте запрос во вкладке **Network**. Сравните:

- `GET /posts` (все посты);
- `GET /posts/1` (один пост).

Сделайте выводы по:

Запрос GET /posts/:

- размеру ответа – 566B

- времени загрузки – 94ms
- количеству записей – 100

Запрос GET /posts/1:

- размеру ответа – 289B
- времени загрузки – 168ms
- количеству записей – 1

Размер, естественно, меньше, а вот время загрузки больше у posts/1, скорее всего из-за того, что post/ был уже загружен и оставалось лишь его показать.

2.5. Работа с параметрами

1. Откройте в браузере:

```
https://jsonplaceholder.typicode.com/posts?userId=1
```

Сравните с `https://jsonplaceholder.typicode.com/posts`:

```
https://jsonplaceholder.typicode.com/posts?userId=1
```

1. сколько объектов вернулось – 10
2. размер ответа – 285B
3. время загрузки – 77ms

https://jsonplaceholder.typicode.com/posts

1. сколько объектов вернулось – 100
2. размер ответа – 350B
3. время загрузки – 59ms

Сделайте вывод, как параметры помогают **уменьшить объём данных**.

Они помогают сделать выборку, и достать только то, что нам нужно.

2.6. Массовые запросы

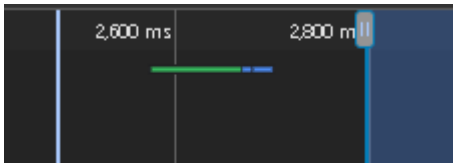
Откройте несколько URL одновременно:

- /posts/1
- /posts/2
- /posts/3

Зафиксируйте:

- как меняется время отклика:

- /posts/1 – 108ms
- /posts/2 – 106ms
- /posts/3 – 85ms
- как браузер параллелит запросы:



По шкале видно, что запросы были выполнены одновременно.

- нет ли избыточных запросов.

Избыточных запросов нет.

2.7. Заголовки и ответы

1. Во вкладке **Headers** выпишите не менее 5 ключевых заголовков, например:
2. Age - 25690
3. Cache-Control - max-age=0
4. ETag - W/"6b80-Ybsq/K6GwwqrYkAsFxqDXGC7DoM"
5. Date - Fri, 06 Nov 2026 15:44:27 GMT
6. Server - cloudflare
7. Сравните вкладки **Preview** и **Response**:

Они показывают одно и то же, но с разной подачей

8. чем они отличаются по представлению данных;

– Preview – JSON который разобран в дерево полей

– Response – сырой текст, как его прислал сервер

9. в каких случаях удобнее использовать Preview, а в каких – Response.

– Preview – когда нужна структура

– Response – когда нужен исходный текст

2.8. Оптимизация запросов

Сравните ответы:

Запрос	Размер ответа	Время загрузки
/posts	262B	33ms
/posts?userId=1	290B	34ms
/posts/1	313B	37ms

Предложите меры оптимизации:

- пагинация (ограничение количества записей на страницу);
- фильтрация (по userId, статусам и т.п.);
- уменьшение размера ответа (убрать лишние поля);
- кэширование на клиенте и сервере;
- использование правильных заголовков (Cache-Control, ETag и т.д.).

Ответ: предлагаю всё, кроме кеширования, так как данные не меняются, их немного и после первого захода они и так отвечают кодом 304.

2.9. Работа с API через Postman

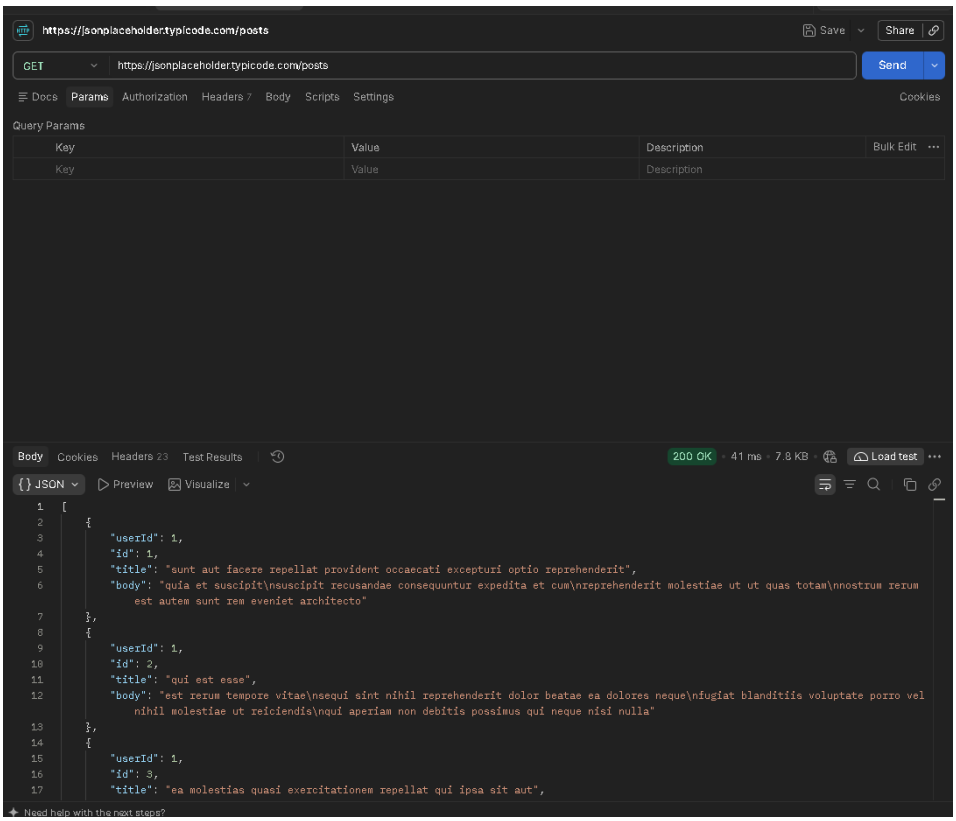
1. Перейдите на сайт:

https://www.postman.com/

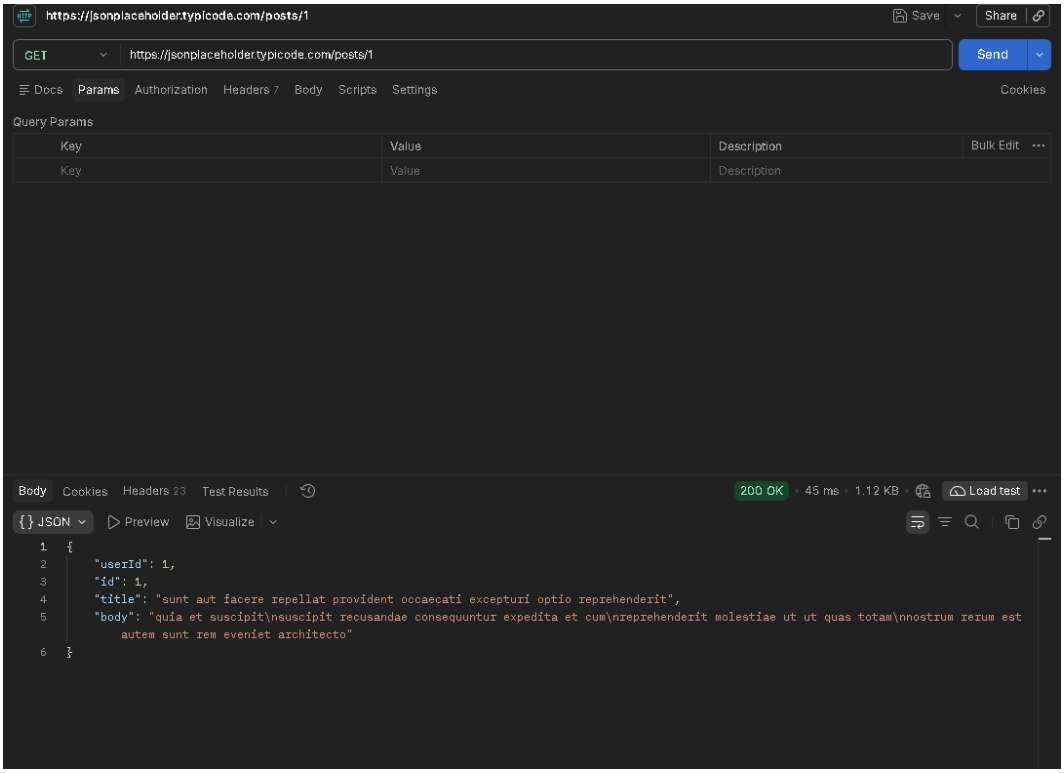
2. Создайте новый workspace.

3. Выполните:

1. GET /posts



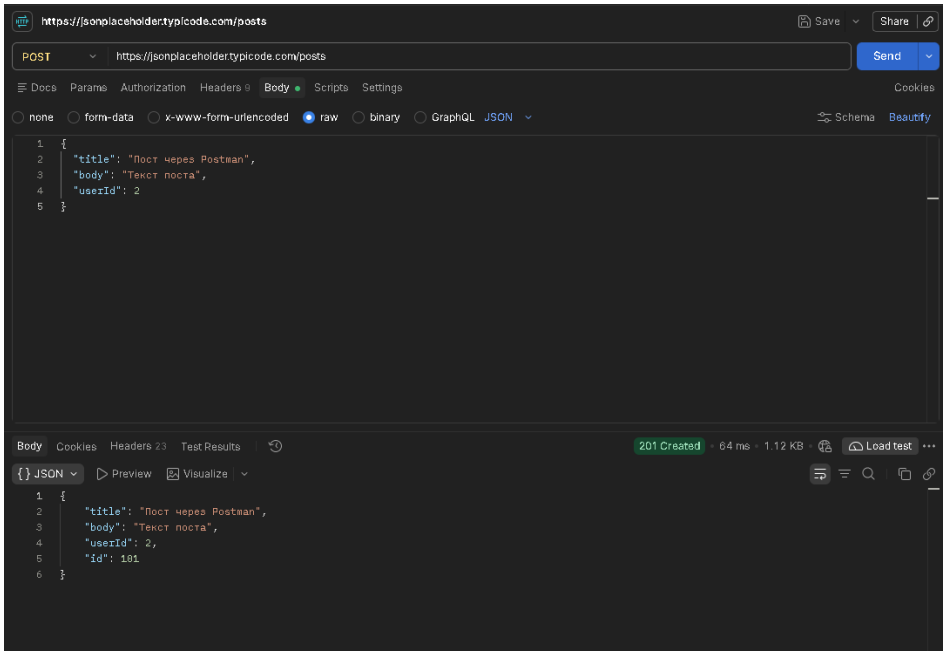
2. GET /posts/1



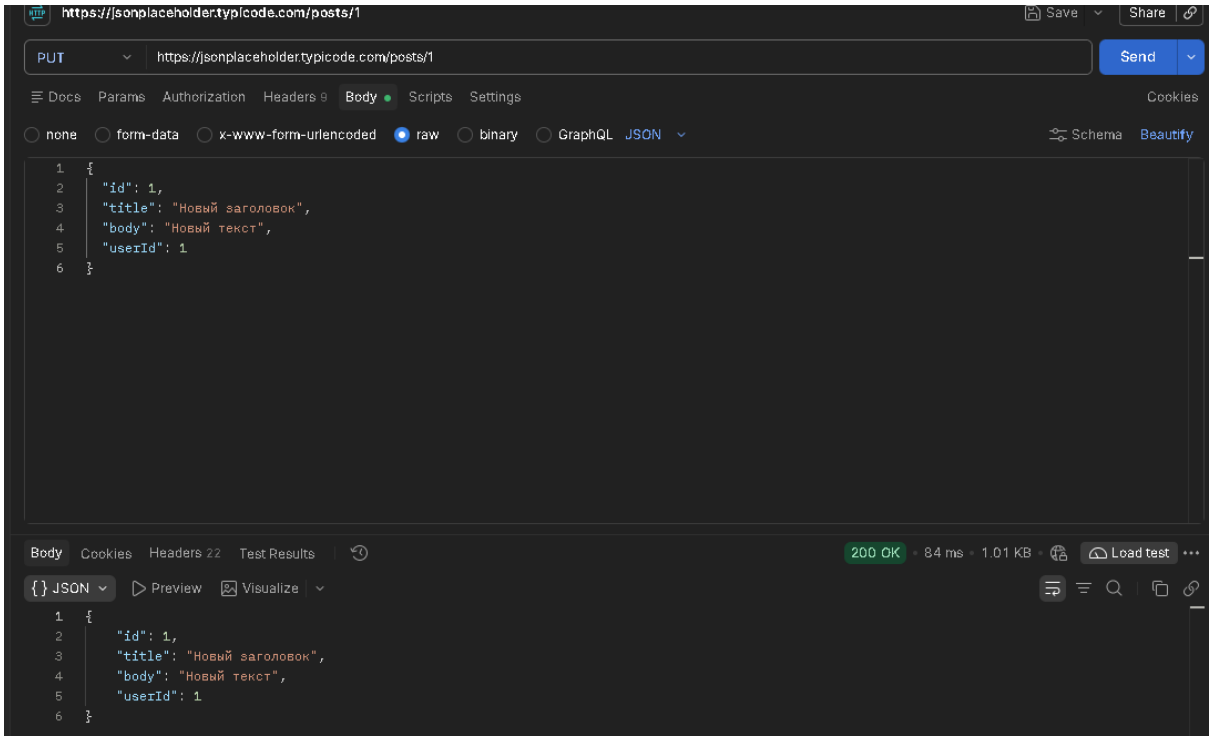
Сравните ответы. Всё как обчно, ответ 200, /post возвращает 100 объектов, а /post/1 возвращает 1, удивительно.

1. Выполните **POST** /posts с JSON-телом:

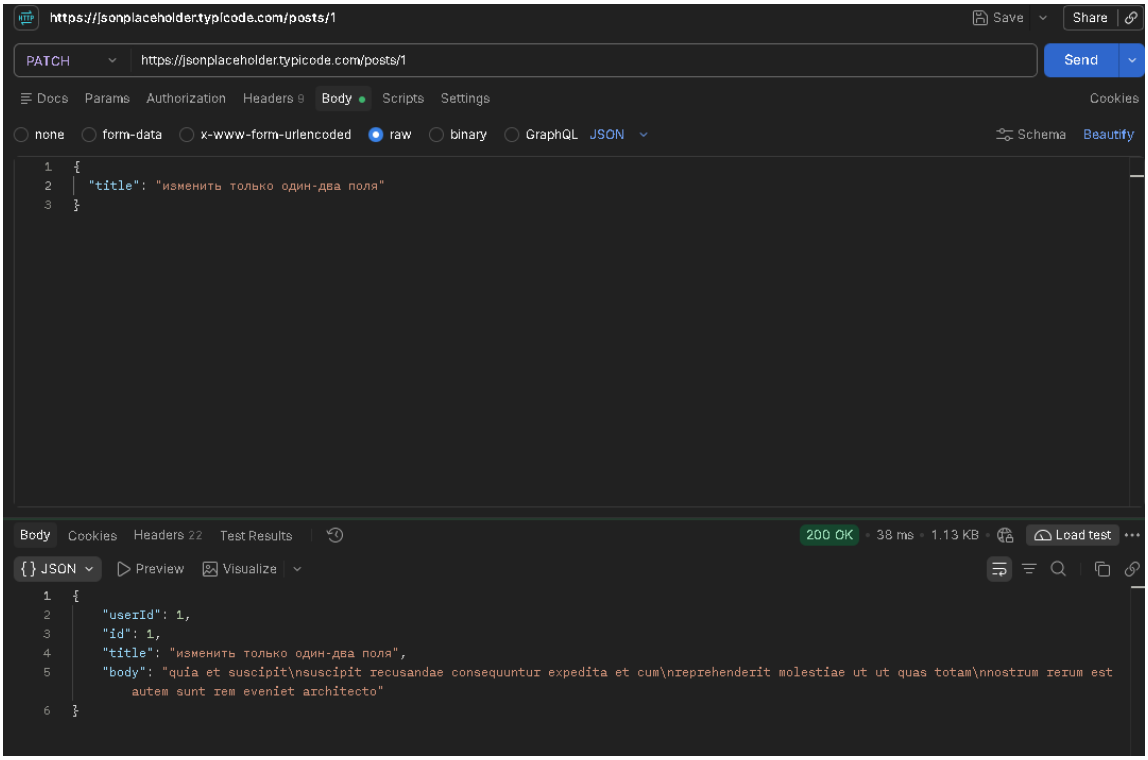
```
{
  "title": "Пост через Postman",
  "body": "Текст поста",
  "userId": 2
}
```



1. Выполните:
2. **PUT** /posts/1 – полное обновление ресурса;



3. **PATCH** /posts/1 – частичное обновление (изменить только один-два поля).



Объясните, чем отличаются результаты.

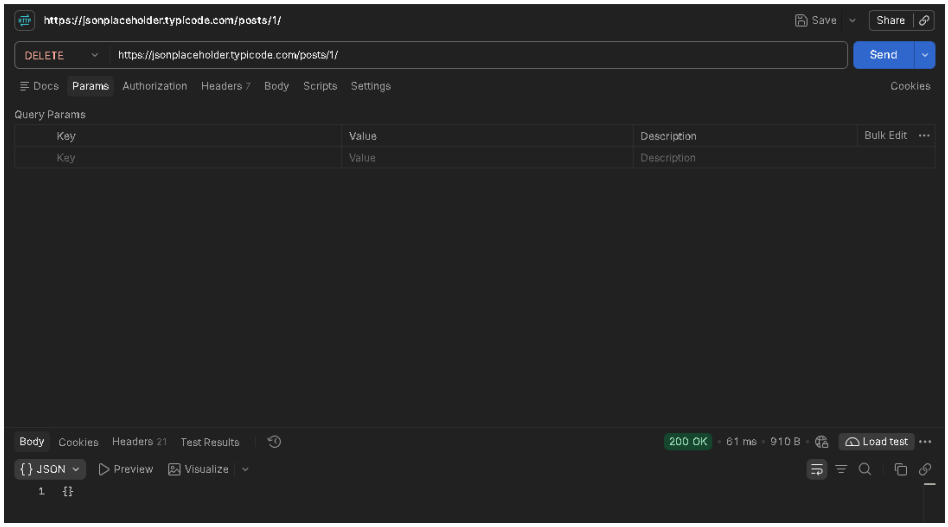
GET – получение записей

POST – добавление записей

PUT – изменение записей

PATCH –тоже изменение, но короче, так как указывать нам надо не все свойства объекта, а только необходимые.

1. Выполните **DELETE** /posts/1 и зафиксируйте:



2. код ответа - 200

3. тело ответа (если есть) – { }.

2.10. Сравнение DevTools и Postman

Сделайте таблицу или краткое сравнительное описание:

Что сравниваем	**DevTools**	**Postman**
Откуда запрос	Его отправляет сама страница, пока сайт открыт	Его собираете вы: метод, URL, заголовки, тело
Что видно по этой работе	Вкладка Network на JSONPlaceholder: posts с кодом **304**, favicon.ico из диска, /posts/1, /posts/2 и /posts/3 ушли параллельно	Тот же GET /posts/1 можно отправить отдельно, без сайта и без кэша браузера
Производительность и кэш	Время, размер, водопад, Cache-Control, ETag, пометка disk cache	Кэша страницы нет. Повторный запрос каждый раз ходит на сервер, поэтому **304** из браузера здесь не виден
Как сохраняется	Живёт, пока открыта вкладка. После закрытия список пропадает	Запрос сохраняется в коллекцию и запускается снова
Проверки	Смотрите, что страница отправила на самом деле	Во вкладке Tests пишется скрипт: код 200, в теле есть id, время меньше порога
Авторизация	Видно заголовок, который страница уже подставила, например Authorization	Токен задаётся в Authorization, подставляется в коллекцию и обновляется скриптом

3. Контрольные вопросы

1. Что такое DevTools и какие задачи решают вкладки Elements, Console и Network?

DevTools — это инструменты браузера для разбора открытой страницы. Elements показывает HTML и CSS и позволяет менять вёрстку на экране. Console показывает ошибки JavaScript и принимает команды. Network показывает запросы страницы: адрес, код, размер, время и заголовки.

2. Как открыть DevTools и какие настройки вкладки Network важны для замеров производительности?

Клавиша F12 или Ctrl+Shift+I, затем вкладка Network. Для замера первой загрузки включите Disable cache, иначе повторный заход возьмёт файл из кэша и время будет около нуля. Оставьте запись включённой, обновите страницу (F5) и смотрите колонки Status, Size и Time. Фильтр All нужен, чтобы не пропали картинки и документ.

3. Перечислите основные части HTTP-запроса (метод, URL/параметры, заголовки, тело) и их назначение.

- Метод — что сделать с ресурсом: прочитать, создать, изменить, удалить.
- URL и параметры — адрес ресурса. Параметры после ? уточняют выборку, например ?userId=1.
- Заголовки — служебные сведения: тип данных, кэш, токен.
- Тело — данные, которые клиент отправляет серверу. У GET его обычно нет.

4. Чем отличаются методы GET, POST, PUT, PATCH и DELETE? Приведите по одному практическому примеру.

- GET читает. GET /posts/1 возвращает одну запись.
- POST создаёт. POST /posts с JSON в теле добавляет запись.
- PUT заменяет ресурс целиком. PUT /posts/1 передаёт все поля записи.

- PATCH меняет только указанные поля. PATCH /posts/1 с одним title обновляет заголовок.
- DELETE удаляет. DELETE /posts/1 удаляет эту запись.

5. Для чего используются методы HEAD и OPTIONS и где встречается preflight-запрос?

HEAD возвращает те же заголовки, что и GET, но без тела: так проверяют размер и ETag, не скачивая файл. OPTIONS сообщает, какие методы и заголовки сервер разрешает. Preflight — это предварительный OPTIONS, который браузер сам шлёт перед «непростым» кросс-доменным запросом, например POST с заголовком Content-Type: application/json на другой сайт. Сервер отвечает, можно ли так обращаться, и только потом уходит основной запрос.

6. Расскажите о группах кодов ответов (1xx–5xx) и приведите примеры для 200, 301/302, 304, 404, 500.

- 1xx — запрос принят, обработка ещё идёт.
- 2xx — успех. 200 — ресурс найден, тело передано.
- 3xx — перенаправление или кэш. 301 — адрес сменился навсегда, 302 — временно. 304 — файл не изменился, тело не присылают, берут копию из кэша.
- 4xx — ошибка на стороне клиента. 404 — такого адреса нет.
- 5xx — сбой на сервере. 500 — сервер не смог обработать запрос.

Примеры из задания:

- 200 – /posts/1 успешно вернул JSON.
- 304 – браузер отправил If-None-Match, сервер ответил «не изменилось», тело не передаётся – экономия трафика.

7. Что такое REST и как ресурс в REST представлен на уровне URL?

REST — способ строить API: данные представлены ресурсами, а действие задаёт метод HTTP. Ресурс — это URL. Коллекция: /posts. Одна запись: /posts/1. Вложенный ресурс: /posts/1/comments. Уточнение выборки: /posts?userId=1.

8. Какие типы данных поддерживает JSON? Приведите короткий пример JSON-объекта и массива.

Ответ:

Типы: string, number, boolean, null, object, array.

```
{
  "id": 1,
  "title": "Hello",
  "published": true,
  "views": 42,
  "tags": ["js", "api"],
  "author": {
    "name": "Ivan",
    "email": null
  }
}
```

9. В чём отличие вкладок Preview и Response в карточке запроса DevTools?

Обе вкладки показывают одно тело. Response — сырой текст, как его прислал сервер: одна строка JSON и символы \n. Preview — то же самое деревом полей, удобно читать userId, id, title, body.

10. Какие заголовки чаще всего анализируются при оптимизации (например, Content-Type, Cache-Control, ETag, Authorization)?

- Content-Type — формат тела, например application/json.
- Cache-Control — сколько времени ответ можно хранить. У JSONPlaceholder это max-age=43200, то есть 12 часов.
- ETag — метка версии. Совпала с сохранённой — сервер отвечает 304 и тело не шлёт.
- Authorization — токен доступа. На открытом JSONPlaceholder его нет, на закрытом API без него будет 401.

11. Какие приёмы снижения нагрузки на сеть можно использовать для API: пагинация, фильтрация, кэширование и др.?

- Пагинация — отдавать страницу, а не все записи: /posts?page=1&_limit=10 вместо 100 записей и 27 КБ.
- Фильтрация — /posts?userId=1 отдаёт 10 записей и около 2,7 КБ вместо 27 КБ.

- Меньше полей — в списке не присылать длинный body, он нужен только в /posts/1.
- Кэш — Cache-Control и ETag на сервере, копия в браузере. Повтор тогда обходится заголовками, как ваши 304 по ~35 мс.

12. Чем работа с API в Postman отличается от анализа запросов в DevTools, и когда уместен каждый инструмент?

DevTools смотрит запросы, которые страница отправила сама: параллельность, кэш, лишний favicon.ico. Запись пропадает вместе с вкладкой. Postman собирает запрос вручную, хранит его в коллекции, гоняет тесты и подставляет токен. DevTools уместен на открытом сайте, когда надо увидеть реальное поведение. Postman уместен, когда API вызывают отдельно от страницы и одни и те же запросы нужно повторять.

Чек-лист для самопроверки

Баллы	Критерии выполнения
7	Выполнены **все пункты практики**: – первичный анализ запросов (таблица + скриншоты); – детальный разбор + сравнение запросов; – работа с параметрами, массовые запросы, заголовки (5+); – таблица оптимизации (`/posts`, `/posts?userId=1`, `/posts/1`); – эксперименты в Postman (GET, POST, PUT, PATCH, DELETE); – сравнение DevTools и Postman; – все обязательные скриншоты (Network, Console, Postman); – есть итоговые выводы и рекомендации; – аккуратное оформление: таблицы, структурированные заголовки, выводы.
6	Выполнена **полная практическая часть** в DevTools (таблицы, скриншоты, сравнения); реализованы **все виды запросов в Postman**: GET, POST, PUT, PATCH, DELETE; есть сравнение DevTools и Postman; составлены рекомендации по оптимизации (5–7 мер);

	оформление есть, но **структура слабее**: не всегда удачно выбраны заголовки, таблицы оформлены неаккуратно.
5	Выполнены основные шаги анализа запросов: таблица с методами, URL, кодами ответа и временем; есть сравнение `/posts` и `/posts/1`, работа с параметрами; сделано 2–3 скриншота; разобраны заголовки и показаны примеры запросов с параметрами; примеры оптимизации приведены, но **не оформлены отдельной таблицей**; в Postman реализованы только **GET/POST**.
3–4	Краткое описание DevTools и HTTP-запросов; выполнен **первичный анализ запросов** (таблица: метод, URL, код ответа, размер, время); сделан **хотя бы один скриншот** из DevTools; есть разбор одного запроса, но **без сравнений и выводов**; JSON/REST API упомянуты, но **Postman не использован** или использован формально.
1–2	Даны лишь **отдельные ответы** без общей структуры; скриншоты отсутствуют или **не соответствуют заданию**; теоретическая часть слабо раскрыта, практическая часть выполнена формально.
0	Работа выполнена **с помощью ИИ** (автоматически сгенерированные ответы без личных скриншотов и анализа); отсутствует выполнение практических шагов, присутствует только копипаст из методических материалов.